



INSTITUTO FEDERAL DA BAHIA
DIRETORIA ACADÊMICA
CURSO TECNOLÓGICO DE ANÁLISE E DESENVOLVIMENTO DE
SISTEMAS

PEDRO LUCAS CARDOSO GOVEIA

ESTRATÉGIAS DE SINCRONIZAÇÃO DE DADOS EM SISTEMAS
OFFLINE-FIRST E PEER-TO-PEER

EUNÁPOLIS - BA

2025

PEDRO LUCAS CARDOSO GOVEIA

**ESTRATÉGIAS DE SINCRONIZAÇÃO DE DADOS EM SISTEMAS
OFFLINE-FIRST E PEER-TO-PEER**

Trabalho de conclusão de curso apresentado como requisito parcial à obtenção do título de Tecnólogo em Análise e Desenvolvimento de Sistemas da Coordenação do Curso de Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia da Bahia.

Orientador: Jurandir da Cruz Barbosa

EUNÁPOLIS - BA

2025

AGRADECIMENTOS

RESUMO

Com o desenvolvimento da computação móvel, projetos são pensados com foco nas qualidades deste tipo de sistema. Em contraste com aplicações web, o estado offline não é um erro ou algo temporário, mas fato que o desenvolvimento móvel deve relevar. Assim o paradigma Offline-First se consolida como uma realidade; este paradigma descreve, dentre outros aspectos, a necessidade de sincronização de dados dos dispositivos que se conectam ao sistema, para manter a consistência e disponibilidade desses dados. Em cenários onde há centralização dos processos, como em um servidor web, existem estratégias específicas para a manutenção dos arquivos e estados; e o mesmo se aplica à computação descentralizada. Redes Peer-to-Peer são uma boa alternativa a um servidor devido o seu baixo custo e privacidade, atualmente ganharam mais tração com o surgimento das blockchains. No entanto, redes Peer-to-Peer apresentam seus próprios desafios de sincronização: peer churn, consistência entre as réplicas e eficiência na sincronização de dados. Desse modo, a necessidade do estudo de estratégias de sincronização em redes Peer-to-Peer em sistemas Offline-First é importante para a reflexão e o direcionamento na escolha da implementação da sincronização de dados congruente ao sistema em produção. Este estudo deve ser realizado com o uso de métricas em ambiente controlado com nós funcionando em rede com conexão descentralizada para obter dados aptos a serem analisados de modo a serem dispostos a comparação das vantagens, limitações e comportamentos dos diferentes tipos de sincronização. No mesmo sentido, o desenvolvimento de uma aplicação Offline-First real para proporcionar substâncias a cada uma dessas aplicações é indispensável para validar as discussões realizadas sobre cada estratégia de sincronização de dados.

Palavras-chave: Offline-First, Peer-to-Peer, Sincronização de dados, CRDT, CAP.

ABSTRACT

With the development of mobile computing, projects are increasingly designed with a focus on the qualities of this type of system. In contrast to web applications, the offline state is not considered an error or a temporary condition, but rather a fact that mobile development must take into account. Thus, the Offline-First paradigm has been consolidated as a reality. This paradigm describes, among other aspects, the need for data synchronization across devices connected to the system, in order to maintain the consistency and availability of such data. In scenarios where processes are centralized, such as on a web server, there are specific strategies for maintaining files and states; the same applies to decentralized computing. Peer-to-Peer networks are a good alternative to servers due to their low cost and privacy, and they have gained further traction with the rise of blockchains. However, Peer-to-Peer networks present their own synchronization challenges: peer churn, replica consistency, and synchronization efficiency. Therefore, studying synchronization strategies in Peer-to-Peer networks within Offline-First systems is essential to guide decision-making in selecting data synchronization implementations that are congruent with the system in production. This study must be carried out using metrics in a controlled environment with nodes operating in a decentralized network, in order to collect data suitable for analysis and comparison of the advantages, limitations, and behaviors of different synchronization strategies. In the same sense, the development of a real Offline-First application to provide substance to each of these strategies is indispensable for validating the discussions conducted about each synchronization approach.

Keywords: Offline-First, Peer-to-Peer, Data Synchronization, CRDT, CAP.

LISTA DE ILUSTRAÇÕES

LISTA DE TABELAS

LISTA DE SIGLAS E ACRÔNIMOS

1. ADS – *Análise e Desenvolvimento de Sistemas*
2. CAP – *Consistency, Availability, Partition Tolerance* – Consistência, Disponibilidade e Tolerância à Partição
3. CmRDT – *Commutative Replicated Data Type* – Tipo de Dado Replicado Baseado em Operações
4. CRDT – *Conflict-free Replicated Data Type* – Tipo de Dado Replicado Sem Conflito
5. CvRDT – *Convergent Replicated Data Type* – Tipo de Dado Replicado Baseado em Estado
6. HTTP – *HyperText Transfer Protocol* – Protocolo de Transferência de Hipertexto
7. IPFS – *InterPlanetary File System* – Sistema de Arquivos Interplanetário
8. OT – *Operational Transformation* – Transformação Operacional
9. P2P – *Peer-to-Peer* – Rede de Pares
10. RAID – *Redundant Array of Independent Disks* – Conjunto Redundante de Discos Independentes
11. RFC – *Request for Comments*
– Solicitação de Comentários (Documentos de Padronização)
12. SQL – *Structured Query Language*
– Linguagem de Consulta Estruturada
13. TCC – *Trabalho de Conclusão de Curso*

14. TCP/IP – *Transmission Control Protocol / Internet Protocol* – Protocolo de Controle de Transmissão / Protocolo de Internet

SUMÁRIO

1	INTRODUÇÃO	1
1.1	OBJETIVOS	2
1.2	JUSTIFICATIVA	2
1.3	ESTRUTURA DO TRABALHO	3
2	METODOLOGIA	3
2.1	ACERCA DAS CATEGORIAS DA PESQUISA	4
2.2	ACERCA DO MÉTODO CIENTÍFICO UTILIZADO	4
3	REFERÊNCIAL TEÓRICO	6
3.1	LINGUAGENS DE PROGRAMAÇÃO	6
3.2	MODELO CLIENTE-SERVIDOR	6
3.3	CONCEITOS ESSENCIAIS PARA A CONSTRUÇÃO DA APLICAÇÃO MÓVEL	6
3.3.1	SISTEMAS <i>OFFLINE-FIRST</i>	7
3.3.2	LINGUAGEM DE PROGRAMAÇÃO: <i>KOTLIN</i>	8
3.3.3	BANCO DE DADOS: SQLite	8
3.4	CONCEITOS ESSENCIAIS DO <i>MIDDLEWARE</i> DE SINCRONIZAÇÃO	8
3.4.1	REDES <i>PEER-TO-PEER</i>	8
3.4.2	LINGUAGEM DE PROGRAMAÇÃO: Go	8
3.4.3	AMBIENTE DE TESTES: Docker	9
3.5	ESTRATÉGIAS DE SINCRONIZAÇÃO	9
3.5.1	TIPOS DE SINCRONIZAÇÃO DE DADOS	9
3.5.2	DESAFIOS NA IMPLEMENTAÇÃO DA SINCRONIZAÇÃO DE DADOS	9
3.5.3	ESTRATÉGIA DE SINCRONIZAÇÃO DE DADOS: CRDT	9
3.5.4	ESTRATÉGIA DE SINCRONIZAÇÃO DE DADOS: OT	9
3.5.5	ESTRATÉGIA DE SINCRONIZAÇÃO DE DADOS: R-SYNC	9
3.5.6	ESTRATÉGIA DE SINCRONIZAÇÃO: ANTI-ENTROPY	9
3.5.7	ESTRATÉGIA DE SINCRONIZAÇÃO: <i>LOG</i> DE OPERAÇÃO	9

3.5.8	ESTRATPEGIA DE SINCRONIZAÇÃO: LWW	9
3.6	TRABALHOS RELACIONADOS	9

1 INTRODUÇÃO

Sistemas de *software* referenciam a infraestrutura disponibilizada pelo ambiente virtual da máquina de executar aplicações; desse modo, esses sistemas incluem o sistema operacional e outros *softwares* utilitários que oferecem serviços fundamentais (TANENBAUM; BOS, 2015). Aplicações, portanto, são *softwares* que agregam funcionalidades adicionais a dispositivos (computadores, celulares, *tablets*, etc.) que o usuário necessita. Assim, Tanenbaum e Van Steen elucidam sobre o uso e a natureza de sistemas distribuídos:

Em muitos casos, os usuários nem percebem que estão lidando com um sistema distribuído. Por exemplo, ao buscar na Web, eles estão acessando uma grande coleção de máquinas geograficamente dispersas que precisam coordenar-se e trocar dados frequentemente. (TANENBAUM; STEEN, 2007)

Estratégias de sincronização de dados são utilizadas em muitas aplicações, devido à necessidade de intercomunicação entre elas, como nas aplicações *web* descritas por Tanenbaum e Van Steen (TANENBAUM; STEEN, 2007). Desse modo, a exploração das soluções existentes, tanto do ponto de vista do uso dos recursos quanto do desenvolvimento de novas ferramentas é uma tarefa desafiadora para os desenvolvedores e útil para o mercado atual. Empresas que atuam na área rural, que utilizam equipamentos com contato intermitente com alguma rede (*internet* ou *intranet*) podem se beneficiar com uso de tecnologias com o perfil *offline*. E esse paradigma de desenvolvimento requer um planejamento sobre a aplicação da sincronização.

Logo, para esse trabalho será aplicada a construção de sistema no paradigma *Offline-First* dentro de uma rede *Peer-to-Peer*, devido aos desafios encontrados nesse tipo de rede para a sincronização das réplicas, assim criando uma rede *Device-to-Device (D2D)*, onde os dispositivos se comunicam entre si de maneira independente. Não será o foco deste trabalho, no entanto, a descrição da instalação da rede e a discussão da construção da aplicação, mas sim a exploração das diferentes estratégias de sincronização e seu comportamento no contexto *Peer-to-Peer* de uma aplicação móvel.

1.1 OBJETIVOS

O objetivo geral deste trabalho é investigar e analisar estratégias de sincronização de dados em redes *P2P* usando métricas para destacar limitações, vantagens e comportamentos em cenários onde existe conectividade intermitente, propondo reflexões e direcionamentos na aplicação das estratégias utilizadas, criando um protótipo funcional de uma aplicação que testa a investigação teórica.

Portanto, os objetivos específicos deste trabalho são:

- Analisar estratégias de sincronização sem considerar a topologia das redes onde são aplicadas;
- Desenvolver um protótipo funcional do sistema com componente de sincronização reutilizável e aplicação móvel para teste;
- Testar o componente de sincronização realizando a discussão sobre a performance com base nas métricas com os dados obtidos.

1.2 JUSTIFICATIVA

Atualmente, com o aumento da computação móvel, existe uma crescente necessidade de sistemas que melhor se adaptam a cenários que não possuem conectividade sem falhas. Sistemas com o paradigma *Offline-First* (construídos de maneira a levar em consideração o funcionamento sem o uso da rede) estão se tornando cada vez mais comuns, pois *Offline-First* não é um erro mas um fato da computação móvel (FEYERKE, 2021). Logo, esse modo de pensar sistemas introduz o desafio da sincronização de dados.

Assim, o estudo de estratégias de sincronização de dados em sistemas *Offline-First* em redes *Peer-to-Peer* oferece oportunidades de exploração de técnicas para solucionar problemas onde: a rede *P2P* monta um ambiente de conexão efêmera, os pares são desconectados e reconectados com frequência e a propagação dos dados é realizada de par-a-par; O sistema projetados com o paradigma *Offline-First* leva em consideração, desde sua criação, dois pontos importantes: a possibilidade de desconexão da rede e a necessidade de sincronização de dados

quando houver a eventual reconexão com a rede.

Esse tipo de aplicação pode ser usada em empresas que trabalham em campo: em atividades florestais, agricultura, avicultura, dentre outras. A utilização de redes *P2P* é interessante pois pode baixar o custo da infraestrutura e, dependendo das opções de *frameworks* adotados, oferecem maior caráter de privacidade para as operações realizadas. Além disso, oferecem os desafios da sincronização para manter a consistência e a disponibilidade dos dados entre as réplicas. Portanto, a experimentação com o desenvolvimento de uma aplicação móvel real nesse contexto oferece oportunidades para testes e reflexões sobre as possibilidades e os cenários que as estratégias melhor se aplicam.

1.3 ESTRUTURA DO TRABALHO

Este trabalho é organizado em introdução, metodologia, referencial teórico, discussão e considerações finais. A introdução apresenta a ideia geral do estudo, incluindo o objetivo geral, os objetivos específicos e a justificativa da pesquisa. A metodologia resume as técnicas, recursos e métricas utilizadas para o desenvolvimento do componente de sincronização (*middleware*, *software* que opera entre componentes comunicantes de um sistema). O referencial teórico contém toda a informação necessária para a compreensão da discussão realizada: conceitos essenciais de sincronização de dados em sistemas distribuídos, conceituação de redes *Peer-to-Peer*, introdução de tecnologias-chave para o desenvolvimento (Go, Kotlin, Docker e libp2p). A discussão consiste na análise dos dados contextualizados pelas métricas destacadas na seção de metodologia em busca de direcionamentos na aplicação das estratégias de sincronização em redes *Peer-to-Peer*. As considerações finais buscam sintetizar o trabalho e propor formas de expandir a discussão.

2 METODOLOGIA

Esta seção apresenta a metodologia usada para o desenvolvimento deste trabalho, isso inclui: a descrição do estudo realizado, o método de construção do trabalho, assim como as métricas utilizadas para a discussão dos resultados dos testes no produto construído.

2.1 ACERCA DAS CATEGORIAS DA PESQUISA

As pesquisas podem possuir diversas categorias na metodologia científica, algumas delas são: quanto a finalidade, quanto a objetivos, quanto a procedimentos e quanto a natureza da abordagem. Desse modo, podemos definir exemplos dessas categorias relevantes para este texto. Logo, quanto a finalidade há a pesquisa aplicada que possui foco na aplicação, utilização e consequências práticas do conhecimento (GIL, 2008). Quanto aos objetivos destaca-se a pesquisa descritiva que tem como objetivo primordial a descrição das características de determinada população ou fenômeno (GIL, 2002). Quanto aos procedimentos: a pesquisa experimental, determina um objeto de estudo e variáveis que são capazes de influenciá-lo e define formas de controle e observação dos efeitos que a manipulação dessas variáveis possuem no objeto. E, quanto a natureza da abordagem é de destaque a qualitativa onde a análise é sistemática e compreensiva, usada usualmente em estudos de caso e pesquisa-ação, pois não há fórmula para orientar o pesquisador, assim também precisa ser maleável (GIL, 2008).

Assim o trabalho se encaixa nas categorias: pesquisa aplicada porque visa a produção de direcionamentos para a implementação das estratégias de sincronização; na descritiva quanto aos seus objetivos de observar os fenômenos (as diferentes estratégias de sincronização em contextos distintos) sem interferir; e, experimental quanto aos procedimentos aplicados, isso é, manipulação dos parâmetros de teste para a simulação de contextos diversos com a coleta de dados qualitativos (GIL, 2008).

2.2 ACERDA DO MÉTODO CIENTÍFICO UTILIZADO

O método científico da pesquisa esclarece os procedimentos lógicos que foram usados na pesquisa, assim, o método escolhido foi o hipotético-dedutivo; este foi escolhido pois define uma abordagem experimental para explicar um fenômeno. Isso se alinha com o objetivo do trabalho de coletar métricas de testes sobre um produto desenvolvido para alcançar um conjunto de postulados que governam os fenômenos analisados na discussão (GIL, 2008). As métricas utilizadas para comparação entre as diferentes estratégias estão nas categorias: acurácia de sincronização, latência e propagação, resiliência da rede e adaptabilidade, eficiência de transmissão de mensagem e utilização de recursos (memória e processamento). Assim as métricas consistem:

- Porcentagem de consistência de dados: porcentagem de *nodes* (ou nós, máquinas na rede que consomem e produzem mensagens) que possuem dados no mesmo estado após um período definido de tempo;
- Porcentagem de resolução de conflitos: a porcentagem em que os conflitos são resolvidos corretamente;
- Latência de sincronização: tempo que a mudança em um node leva para alcançar os demais;
- Tolerância de *peer churn* (conexão e desconexão de nodes na rede): degradação de performance com a conexão e desconexão de nodes frequentes;
- Tempo de recuperação: tempo que leva para ressincronizar após falhas;
- Excedente de mensagem: razão de mensagens de sincronização para dados atualizados;
- Uso de banda larga por *node*: quantidade de banda larga usada por node por sincronização;
- Utilização de memória: uso de memória em *buffers* e metadados;
- Uso de processamento: processamento de mensagens e resolução de conflitos.

Essas métricas vão ser acessadas através de testes realizados em uma rede, simulando um contexto *Peer-to-Peer*. Assim, para o ambiente de simulação de *nodes* cabe observar os seguintes casos para a coleta de dados: atualização de dados, resolução de conflitos, publicar ou convergir estados (em busca de consistência); Tais observações testam a capacidade do sistema de manter a integridade do fluxo e manter a latência dentro dos limites aceitáveis. Em cenários reais há situações que levam a perda de pacotes, exemplos desses sendo: alta latência e *peer churn* (NASRALLAH et al., 2018); assim, na construção dos testes foi implementada aleatoriedade do contexto da rede para simular cenários reais, juntamente com o agrupamento de *nodes* em subredes.

3 REFERÊNCIAL TEÓRICO

Esta seção apresenta conceitos necessários para o desenvolvimento do trabalho, sendo esses: definição de conceitos básicos (linguagens de programação e modelo cliente-servidor) descrição das tecnologias usadas para a construção da aplicação (*Go*, *Kotlin*, redes *Peer-to-Peer*), diferentes tipos de estratégias de sincronização e tópicos essenciais para o seu entendimento tão como comparações com trabalhos relacionados.

3.1 LINGUAGENS DE PROGRAMAÇÃO

A linguagem nativa de computadores é a binária (uns e zeros), todas as instruções e dados devem ser providas nesse formato para serem processados. Código binário nativo é chamado de linguagem de máquina. Com o passar do tempo foram aparecendo novas formas de construir programas; sucedendo o código de máquina os programas começaram a serem escritos em sequências de números hexadecimais. Em seguida o surgimento linguagens *assembly*, os códigos então começaram a ser escritos usando uma série de comandos mnemônicos que representam um conjunto de instruções em código de máquina. Essas linguagens de programação não são apropriadas para o desenvolvimento de aplicação em larga escala, portanto, o surgimento das linguagens de alto nível como C. Estas definem um conjunto de mecanismos que servem como abstrações das instruções de máquina (*loops*, condicionais, funções, etc) (BAILEY, 2005).

Construir estruturas de dados e algoritmos requer uma linguagem de programação de alto nível (GOODRICH; TAMASSIA; MOUNT, 2011). Logo, para o desenvolvimento da aplicação móvel e dos *middlewares* de sincronização foram, portanto, escolhidas as linguagens de programação *Kotlin* e *Golang*, respectivamente.

3.2 MODELO CLIENTE-SERVIDOR

3.3 CONCEITOS ESSENCIAIS PARA A CONSTRUÇÃO DA APLICAÇÃO MÓVEL

A aplicação consiste em um leitor de *ePub* com a possibilidade de fazer anotações, a sincronização será feita a partir dos compartilhamento dessas anotações na rede, todos os

pares deveram ter as anotações dos demais. Desse modo, as tecnologias relevantes são: *Go* e *Kotlin* para linguagens de programação e *SQLite* para banco de dados local, onde os dados serão armazenados quando no dispositivo celular do usuário.

3.3.1 SISTEMAS *OFFLINE-FIRST*

Offline-first é um paradigma de programação que especifica, dentre outros aspectos, o comportamento de aplicações em relação a falta de conectividade (*OFFLINE FIRST COMMUNITY*, n.d.). Um bom exemplo desse paradigma são os *Progressive Web Apps (PWAs)*, para uma aplicação ser um *PWA* esta deve englobar conceitos que formulam o modo que foi construída; dentre eles destaca-se, para esse trabalho, a independência de conexão (*BIØRN-HANSEN; MAJCHRZAK; GRØNLI*, 2018). A arquitetura define, também, o armazenamento local de dados como componente essencial para sua implementação, e com ele a sincronização e a resolução de conflitos. *Offline-first* assume a existência da necessidade de conexão com alguma rede, sendo essa efêmera, e especifica como aplicações devem funcionar nesse contexto (*GUDA*, 2025).

A oferta variável de recursos da computação móvel sugere que aplicações para esse contexto devem ser adaptáveis. Historicamente, adaptação é a troca do recurso por outro ou pela qualidade do serviço prestado, isso é: técnicas de redução, filtragem e transformação de dados para trafegarem na rede; o processo de adaptabilidade é disparado automaticamente após a leitura do contexto. Contexto é toda informação relevante para aplicação que define seu comportamento; desse modo, as informações de contexto podem ser categorizadas em: físicas, relativas ao *hardware*; lógicas, relativas ao *software*; e, sociais, relativas à comunidade de usuários e seu contexto (*AUGUSTIN et al.*, 2001). A necessidade por uma funcionalidade *offline* robusta persiste, como em sistemas de *chat* que funcionam em *intranets* de ambientes isolados (*GUDA*, 2025), por exemplo, em áreas rurais de plantio, eventos em locais remotos e navios de cruzeiro.

3.3.2 LINGUAGEM DE PROGRAMAÇÃO: *KOTLIN*

3.3.3 BANCO DE DADOS: *SQLite*

3.4 CONCEITOS ESSENCIAIS DO *MIDDLEWARE* DE SINCRONIZAÇÃO

3.4.1 REDES *PEER-TO-PEER*

3.4.2 LINGUAGEM DE PROGRAMAÇÃO: *Go*

Go é uma linguagem de programação simimilar a linguagem C, dela herdou a sintáse de expressões, tipos de dados básicos e a ênfase em programas que compilam para código de máquina eficiente. Criada por Robert Griesemer, Rob Pike, e Ken Thompson, todos da *Google*, em 2007 para resolver o problema crescente da complexidade dos sistema da empresa. *Go* é apropriado para a contrução de servidores que atuam como infraestutura em rede (DONOVAN; KERNIGHAN, 2016), portanto a escolha para a construção do *middleware*.

A linguagem coloca muita ênfase na eficiência, *Go* possui funcionalidades inspira-das em outras linguagens, como: *garbage collection* de linguagens como *Java*, o conceito de pacotes de *Modula-2*, a sintaxe de métodos de *Oberon-2*, dentre outras funcionalidades. As *goroutines*, fluxos leves de execução, são outro aspecto da língua que demonstra a preocupação por eficiência, onde: criar uma *goroutine* é barato e criar milhões é prático (DONOVAN; KERNIGHAN, 2016).

Alguns exemplos de produtos que usam *Go* são: *Traefik*, um *proxy* reverso e balance-ador de carga, isso é, simplifica o roteamento entre vários serviços (microserviços e aplicações em *Docker*) (TRAEFIK LABS, 2025); *Caddy*, plataforma para servir *sites*, aplicações e serviços (CADDY SERVER, 2025); *KrakenD*, simplifica acesso à mutiplos serviços de *backend*, ele faz isso por centralizar, acelerar e proteger as conexões (KRAKEND, 2025).

3.4.3 AMBIENTE DE TESTES: Docker

3.5 ESTRATÉGIAS DE SINCRONIZAÇÃO

3.5.1 TIPOS DE SINCRONIZAÇÃO DE DADOS

3.5.2 DESAFIOS NA IMPLEMENTAÇÃO DA SINCRONIZAÇÃO DE DADOS

3.5.3 ESTRATÉGIA DE SINCRONIZAÇÃO DE DADOS: CRDT

3.5.4 ESTRATÉGIA DE SINCRONIZAÇÃO DE DADOS: OT

3.5.5 ESTRATÉGIA DE SINCRONIZAÇÃO DE DADOS: R-SYNC

3.5.6 ESTRATÉGIA DE SINCRONIZAÇÃO: ANTI-ENTROPY

3.5.7 ESTRATÉGIA DE SINCRONIZAÇÃO: *LOG* DE OPERAÇÃO

3.5.8 ESTRATÉGIA DE SINCRONIZAÇÃO: LWW

3.6 TRABALHOS RELACIONADOS

REFERÊNCIAS

- AUGUSTIN, Iara et al. Requisitos para o projeto de aplicações móveis distribuídas. Versão portuguesa. In: ANAIS do VII Congreso Argentino de Ciencias de la Computación (CACIC). [S.l.: s.n.], out. 2001. Origem: Red de Universidades con Carreras en Informática (RedUNCI). Data de exposição/publicação: outubro 2001.
- BAILEY, Tim. **An Introduction to the C Programming Language and Software Design**. [S.l.: s.n.], 2005. Draft 0.6, July 12, 2005. Material didático utilizado em curso de Engenharia de Software.
- BIØRN-HANSEN, Andreas; MAJCHRZAK, Tim A.; GRØNLI, Tor-Morten. Progressive Web Apps for the Unified Development of Mobile Applications. In_____. **Web Information Systems and Technologies (WEBIST 2017)**. Cham: Springer, 2018. v. 322. (Lecture Notes in Business Information Processing), p. 64–86. DOI: 10.1007/978-3-319-93527-0_4.
- CADDY SERVER. **Caddy Documentation**. [S.l.: s.n.], 2025.
<https://caddyserver.com/docs/>. Acesso em: 09 dez. 2025.
- DONOVAN, Alan A. A.; KERNIGHAN, Brian W. **The Go Programming Language**. Boston, MA: Addison-Wesley Professional, 2016. First printing October 2015. ISBN 978-0-13-419044-0.
- FEYERKE, Alex. **Offline-first: Banco de dados no front-end**. YouTube. 2021. Disponível em: https://www.youtube.com/watch?v=dPz_5-MEvcg. Acesso em: 25 mai. 2025.
- GIL, Antonio Carlos. **Como elaborar projetos de pesquisa**. 4. ed. São Paulo: Atlas, 2002. ISBN 85-224-3169-8.
- _____. **Métodos e técnicas de pesquisa social**. 6. ed. São Paulo: Atlas, 2008.
- GOODRICH, Michael T.; TAMASSIA, Roberto; MOUNT, David M. **Data Structures and Algorithms in C++**. 2. ed. Hoboken, NJ, USA: John Wiley & Sons, Inc., 2011. ISBN 978-0-470-38327-8.
- GUDDA, Varun Reddy. Implementation of Offline-First Architectures for Android Internet-Based Chat Systems. **International Journal of Multidisciplinary Research and Growth Evaluation**, v. 6, n. 3, p. 1200–1203, mar. 2025. ISSN 2582-7138. DOI: 10.54660/IJMRGE.2025.6.3.1200-1203.
- KRAKEND. **KrakenD API Gateway Documentation**. [S.l.: s.n.], 2025.
<https://www.krakend.io/>. Acesso em: 09 dez. 2025.

NASRALLAH, Ahmed et al. Ultra-Low Latency (ULL) Networks: The IEEE TSN and IETF DetNet Standards and Related 5G ULL Research. **arXiv preprint arXiv:1803.07673**, 2018. Survey comprehensively covers link and network layer latency reduction standards and studies up to July 2018.

OFFLINE FIRST COMMUNITY. **Offline First**. n.d. Disponível em: <https://offlinefirst.org/>. Acesso em: 13 dez. 2025.

TANENBAUM, Andrew S.; BOS, Herbert. **Modern Operating Systems**. 4. ed. Upper Saddle River: Pearson, 2015.

TANENBAUM, Andrew S.; STEEN, Maarten van. **Sistemas distribuídos: princípios e paradigmas**. 2. ed. Upper Saddle River: Pearson Prentice Hall, 2007. Disponível em: <https://csc-knu.github.io/sys-prog/books/Andrew%20S.%20Tanenbaum%20-%20Distributed%20Systems.%20Principles%20and%20Paradigms.pdf>. Acesso em: 20 jul. 2025.

TRAEFIK LABS. **Traefik Proxy Documentation**. [S.l.: s.n.], 2025. <https://traefik.io/>. Acesso em: 09 dez. 2025.